

```
openapi: 3.0.3
info:
  title: Events API
  version: 1.0.0
```

```
paths:
  /events:
    get:
      summary: list
      operationId:
        listEvents
      responses:
        '200':
          schema:
            $ref: '#/Event'
```

```
components:
  schemas:
    Event:
      type: object
      properties:
        name: string
```

SDD

12º MEETUP ITU DEVELOPERS · 11.10.2025



Spec Driven Design

Deixe a Spec Guiar o Código — e o Copilot

#OpenAPI

#GitHubCopilot

#SpecDrivenDesign

#dotnet

Gustavo Bellini Bigardi · CTO & Consultor @ Clarus IT

Quem sou eu?



Gustavo Bellini Bigardi

CTO & Consultor · Clarus IT

about.me/gbbigardi
github.com/gustavobigardi

Vamos falar sobre

- ▶ O problema de desenvolver sem especificação
- ▶ O que é Spec Driven Design
- ▶ Como isso multiplica a qualidade do GitHub Copilot
- ▶ Demo ao vivo: spec → API C# .NET em tempo real

Levanta a mão...



Quem já precisou integrar com uma API sem documentação?



Quem já foi responsável por essa API sem documentação?

(Sério, levanta mesmo)

A história de sempre...

50

engenheiros

30

APIs internas

0

documentação

- Times reconstruindo endpoints do zero
- Reuniões para descobrir "o que essa API retorna"
- Integrações quebrando silenciosamente

"Isso é um extremo. Mas quem aqui nunca trabalhou com uma API que mudou sem aviso?"

O problema não é preguiça.

O processo não está configurado para manter spec e código **juntos**.

Os 4 Padrões de Desenvolvimento de API

Padrão	Como funciona	Resultado
❌ Code-first + "documento depois"	"A gente coloca no Swagger depois"	Depois não chega
❌ Code-first + anotações	[SwaggerOperation] no código	Diverge com o tempo
❌ Design-first → abandona	Spec no cloud, código no Git	Duas fontes de mentira
✅ Design-first → evolui junto	Spec no mesmo repo que o código	Contrato real

Duas Fontes de Mentira



Spec

Stoplight / SwaggerHub — desatualizada



Código

GitHub — foi além da spec há meses

A spec que não reflete o código e o código que não reflete a spec — ambas são inúteis.

O que é Spec Driven Design?

Uma especificação **legível por máquina** é criada **antes** de qualquer código. A spec se torna a **fonte única da verdade** da qual tudo é derivado.



Endpoints

e métodos HTTP



Schemas

request e response



Autenticação



Erros

respostas tipadas



Exemplos

de dados reais

OpenAPI — A Linguagem da Spec

 openapi.yaml

```
openapi: 3.0.3
info:
  title: Meetup Events API
  version: 1.0.0
paths:
  /events:
    get:
      summary: Lista todos os eventos
      operationId: listEvents
      responses:
        '200':
          schema:
            $ref: '#/components/schemas/Event'
```

Um arquivo YAML (ou JSON) no seu repositório Git. **É código. Tem review. Tem versão.**

Uma Spec → Muitas Ferramentas

O que você ganha	Ferramenta	Custo
 Documentação interativa	Swagger UI · Redoc · Scalar	Grátis
 Mock server imediato	Stoplight Prism · Mockoon	Grátis
 Testes de contrato	Dredd · Specmatic	Grátis
 Geração de código	OpenAPI Generator	Grátis
 Linting / qualidade	Spectral	Grátis

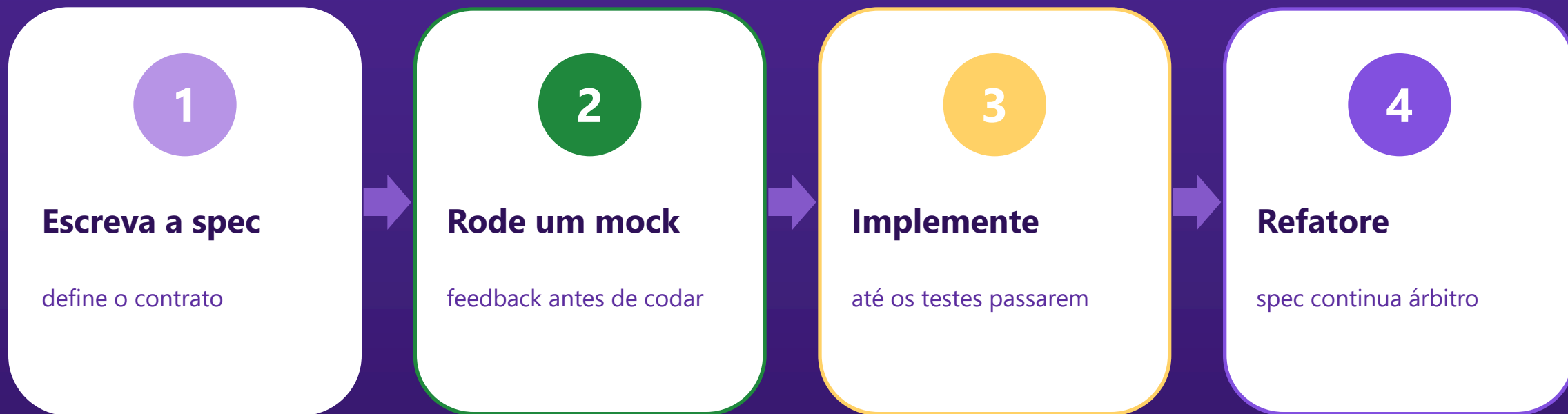
Escreva o YAML **uma vez**. Ganhe tudo isso **automaticamente**.

SDD, TDD e BDD — Irmãos de Filosofia

Metodologia	Especificação	Quando	Audiência
TDD	Testes unitários	Antes da função	Devs
BDD	Given / When / Then	Antes da feature	Negócio + Devs
SDD	OpenAPI / JSON Schema	Antes da API	Times + Consumidores

*No TDD você escreve o teste antes do código. No SDD você escreve o **contrato** antes de tudo.*

O Ciclo do SDD



⌚ A spec é a fonte da verdade em todo o ciclo

Por Que Isso Importa AGORA?



Em 2026, todo mundo está usando **GitHub Copilot**.

E o Copilot **mudou completamente** essa equação.

Como o Copilot Funciona

O Copilot monta um contexto para o modelo de IA a partir de:

Neighboring tabs

Todos os arquivos abertos no IDE

Fill-in-the-Middle

Código antes e depois do cursor

Chat history

Histórico da conversa

Contexto rico = previsões melhores.
É simples assim.

Spec Aberta = Contexto Perfeito

✗ Sem Spec	✓ Com Spec Aberta
Chuta tipos dos parâmetros	Lê tipos exatos do JSON Schema
Inventa formato de resposta	Segue o schema de resposta
Ignora validações	Usa constraints: min / max / pattern
Inventa tratamento de erros	Segue schemas de erro documentados
Nomeia coisas genericamente	Usa operationId e nomes do schema

Abra a spec no VS Code ao lado do código. O Copilot pega o contexto **automaticamente**.

O Truque: copilot-instructions.md

```
.github/copilot-instructions.md
```

Spec Driven Design

Todos os endpoints devem seguir o contrato em `./openapi.yaml`.

Para cada endpoint implementado:

- Valide o request contra o schema da spec
- Retorne respostas tipadas conforme o schema
- Use os `operationId` como nomes dos métodos
- Implemente os códigos de erro documentados

Lido **automaticamente** pelo Copilot em todos os chats do repositório.

Modelos — Sem Gastar Muito

Copilot Free

R\$ 0

Modelos incluídos

GPT-5 mini

50 premium requests/mês · 2.000 completions

Copilot Pro

~R\$ 55/mês

Modelos incluídos

GPT-5 mini · GPT-4.1 · GPT-4o

300 premium requests/mês · completions ilimitados

Atenção: precificação muda em 1/junho/2026



GitHub muda para cobrança por AI Credits/tokens. Pro segue em US\$ 10/mês com US\$ 10 em créditos + US\$ 5 flex.

<https://github.blog/news-insights/company-news/github-copilot-individual-plans-introducing-flex-allotments-in-pro-and-pro-and-a-new-max-plan>

DEMO AO VIVO



Spec → API C# .NET

com GitHub Copilot

DOMÍNIO

API de eventos e ingressos

STACK

C# .NET 10 · VS Code · GPT-5-mini

MODELOS

Incluídos no Copilot e Copilot Pro

O que Vamos Construir

Meetup Events API — uma API para gerenciar eventos como este 🎉

Entidades

Event

- id, name, description, date
- location, capacity, availableSpots
- ticketPrice, status

Ticket

- id, eventId, buyerName
- buyerEmail, purchaseDate, status

Endpoints

- **GET** /events
- **POST** /events
- **GET** /events/{id}
- **PUT** /events/{id}
- **DELETE** /events/{id}
- **GET** /events/{id}/tickets
- **POST** /events/{id}/tickets

FASE 1 · 8 min

Criando a Spec

- 1 Criar estrutura do projeto
- 2 Criar copilot-instructions.md — as regras para o Copilot
- 3 Criar openapi.yaml — começar o cabeçalho, Copilot completa
- 4 Usar o Chat para gerar schemas e endpoints completos

terminal

```
$ mkdir meetup-events-api
$ cd meetup-events-api
$ mkdir .github
$ touch openapi.yaml
$ touch .github/
    copilot-instructions.md
$ code .
```

Resultado: openapi.yaml completo, com schemas, paths e exemplos

FASE 2 · 12 min

Gerando o Código C#

1

Criar o projeto .NET

`dotnet new webapi -n EventsApi --no-openapi`

2

Abrir openapi.yaml ao lado do código

← *o segredo*

3

Pedir ao Copilot Chat para gerar

Models, Controllers, registrar no Program.cs

4

Mostrar o resultado compilando



Como Começar Hoje

1

Escolha o tier

Free (R\$ 0) para aprender · Pro (~R\$ 55/mês) para uso frequente

2

Ferramentas para a spec

editor.swagger.io · VS Code + extensão OpenAPI

3

Aplique as 3 regras de ouro

Spec no repo · Exemplos em tudo · Spec como código

As 3 Regras de Ouro



Spec no repositório

Mesmo repo, mesmo PR, mesmo code review que o código



Exemplos em tudo

Schemas sem exemplos são inúteis para AI e para mocks



Spec como código

Mudança na API = mudança na spec = review do time

Resumo



SDD = TDD para APIs

Defina o contrato antes do código



OpenAPI no repositório

Contexto gratuito e perfeito para o Copilot



GPT-5-mini (incluso no plano gratuito e Pro)

Mais que suficiente para o dia a dia



Spec lidera. Código segue.

Obrigado! 🙌

Materiais da palestra

<https://github.com/gustavobigardi/meetup-itu-developers-spec-driven-design>

